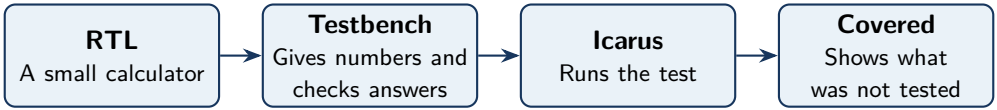
A Very Simple RTL Coverage Example

A 4-bit calculator with Icarus Verilog + Covered

Two inputs, one operation choice, and one result.

The Main Idea



Question answered by coverage

The test passed, but did the test try every operation and enough input values?

The Complete RTL

```
module simple_alu (
  input  wire [3:0] a,
  input  wire [3:0] b,
  input  wire [1:0] op,
  output reg  [3:0] result
);

  always @* begin
    case (op)
      2'b00: result = a + b;  // ADD
      2'b01: result = a & b;  // AND
      2'b10: result = a | b;  // OR
      2'b11: result = a ^ b;  // XOR
    endcase
  end
endmodule
```

Signals

- a: first number
- b: second number
- op: operation choice
- `result`: answer

op	Action
00	ADD
01	AND
10	OR
11	XOR

The Testbench Runs Two Tests

Test 1: ADD

a=3, b=2, op=00
Expected result: 5

Test 2: AND

a=6, b=3, op=01
Expected result: 2

Not tested

op=10 (OR) and op=11 (XOR)

```
initial begin
  // Test ADD: 3 + 2 = 5
  a = 4'h3;
  b = 4'h2;
  op = 2'b00;
  #1;
  if (result != 4'h5)
    $display("ERROR: ADD");

  // Test AND: 6 & 3 = 2
  a = 4'h6;
  b = 4'h3;
  op = 2'b01;
  #1;
  if (result != 4'h2)
    $display("ERROR: AND");
end
```

Real Commands: Compile and Simulate

Run these commands inside the `coverage_exp` directory.

```
$ iverilog -g2001 -Wall -s simple_alu_tb \
    -o build/simple_alu.vvp \
    src/simple_alu.v tb/simple_alu_tb.v
$ vvp build/simple_alu.vvp
```

Real output from this example:

```
VCD info: dumpfile build/simple_alu_raw.vcd opened for output.
PASS: ADD and AND tests passed
tb/simple_alu_tb.v:56: $finish called at 5 (1ps)
```

What this means

Icarus compiled the RTL and testbench. The two checked answers were correct, and the simulation wrote signal activity to a VCD file.

Real Commands: Create the Coverage Report

```
$ sed '/^$comment /d' build/simple_alu_raw.vcd > build/simple_alu.vcd

$ covered score -t simple_alu -i simple_alu_tb.dut \
    -v src/simple_alu.v -vcd build/simple_alu.vcd \
    -o build/simple_alu.cdd

$ covered report -m ltc -d s \
    -o build/coverage_summary.rpt build/simple_alu.cdd

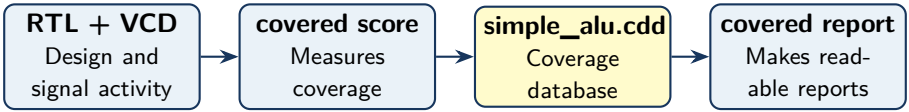
$ covered report -m ltc -d v -i \
    -o build/coverage_detail.rpt build/simple_alu.cdd
```

Real output from Covered:

```
Covered covered-0.7.10 -- Verilog Code Coverage Utility
Reading design...
Scoring VCD dumpfile build/simple_alu.vcd...
***  Scoring completed successfully!  ***
```

The first command removes one VCD comment that old Covered 0.7.10 cannot read from Icarus Verilog 13 output.

What Is the .cdd File?



CDD in simple words

The CDD file stores the design structure and the measured hit/miss data. It is the saved result of `covered score`.

- `VCD`: what the signals did during simulation.
- `CDD`: which coverage points were hit or missed.
- `RPT`: text made from the CDD for people to read.

Three Simple Coverage Questions

Coverage	Simple question	This example
Line	Did the test run each RTL line?	OR and XOR lines were missed.
Toggle	Did each input/output bit change both ways?	Only a few values were used.
Logic	Did we try different 0/1 input choices?	One ADD and one AND are not enough.

Why only three types?

This RTL has no clock, state machine, or memory. Keeping those features out makes the first coverage example easier to follow.

More about code coverage: chipverify.com/verification/code-coverage

Line Coverage: Which RTL Lines Ran?

```
always @* begin
2'b00:  result = a + b; // ADD
2'b01:  result = a & b; // AND
2'b10:  result = a | b; // OR
2'b11:  result = a ^ b; // XOR
```

Green = executed Red = not executed

Real result

Hit: 3
Miss: 2
Total: 5

3/5 = 60%

Why not count case?

It selects a path; Covered counts the executable lines in the paths.

Toggle Coverage: Did Bits Change Both Ways?

Bits measured

`a`	4 bits
`b`	4 bits
`op`	2 bits
`result`	4 bits
Total	**14 bits**

Each bit gets two questions:

- Did it change from 0 to 1?
- Did it change from 1 to 0?

Example: op

Test sequence: 00 → 00 → 01

`op[0]`: 0 → 1 seen
`op[0]`: 1 → 0 missing
`op[1]`: stays at 0

Real result for all 14 bits

0 → 1: **9/14 = 64%**
1 → 0: **3/14 = 21%**

Logic Coverage: Which Input Choices Were Tried?

Covered treats an expression input as **0** or **non-zero**.

Coverage point	Choices	Count
`@*` event	triggered	1
`case(op)` decision	false, true	2
`a + b`	00, 01, 10, 11	4
`a & b`	0-, -0, 11	3
`a | b`	1-, -1, 00	3
`a ^ b`	00, 01, 10, 11	4
Total		**17**

A dash in `0-` means the other input may be either value.

ADD example

Initial values `a=0, b=0` try **00**.
ADD values `a=3, b=2` try **11**.
Choices **01** and **10** are missing.

Real result

Hit: 6 Miss: 11 Total: 17

6/17 = 35%

Real Output: Coverage Results

From `build/coverage_summary.rpt`:

```
LINE COVERAGE
  simple_alu  simple_alu.v   3/  2/  5   60%

TOGGLE COVERAGE
  simple_alu  simple_alu.v   9/  5/ 14   64%
                             3/ 11/ 14   21%

LOGIC COVERAGE
  simple_alu  simple_alu.v   6/ 11/ 17   35%
```

From `build/coverage_detail.rpt`:

```
Missed Lines
  16: result = a | b
  17: result = a ^ b
```

The important result

The test passed, but OR and XOR were never tested.

How to Improve the Test

1. Add one OR test with `op=10`.
2. Add one XOR test with `op=11`.
3. Try more values for `a` and `b`.
4. Run `make report` again.

Main lesson

The testbench checks whether answers are correct. Coverage shows which RTL operations and signal values the testbench did not try.